ProtocolShift: An Empirical Framework for REST vs. gRPC Protocols, Isolating Protocol Metrics Across Multi-Paradigm Cloud (AWS) Database Architectures

Shlok Doshi (B039), Moksh Jain (B054), Aranck Jomraj (B057), Kahaan Kakabalia (B058)

B.Tech Computeer Engineering, B Divsion, MPSTME

Abstract

The present paper describes ProtocolShift, an experimental testbed based on high concurrency and aimed at isolating the performance difference between HTTP/1.1 REST and gRPC by separating transport and serialization overheads and application logic and I/O latency. The study employs a multi-database topology (PostgreSQL, Redis, and MongoDB) to rigorously test the performance of the so-called serialization tax, head-of-line blocking, and critical p95/p99 tail latencies and ensures that the test is conducted at extreme computational load. Comparing JSON over HTTP/1.1 with multiplexed HTTP/2 using Protocol Buffers, the study determines whether the binary efficiency of gRPC exceeds the persistent storage bottlenecks in different data paradigms. The results lead to a framework of migrating measurements based on the Strangler Fig pattern which offers a rigorous and empirical basis of selecting and optimizing protocols in scalable distributed architectures.

Index Terms

Distributed Systems, gRPC, REST, Observability, Tail Latency, Database Architecture

I. Introduction

Microservices and service-oriented architectures (SOA) have led to the discovery of Remote Procedure Call (RPC) or RESTful architecture as an engineering decision with colossal ramifications in the scalability of the system and resource allocation. Despite Rests reliance on the industry standard HTTP/1.1 protocol and JavaScript Object Notation (JSON) data formats, which have long been de facto in distributed communication between services [3], it is now facing demands to support high performance. The overhead of performing serialization on the verbose nature of the default state of JSON at any time, combined with the overhead of text-parzing the default state of the same can be up to 10-40 percent of the total response time at peak load conditions. On the other hand, gRPC, making use of the binary multiplexing of HTTP/2 and the tightly packed schema-driven structural contracts of Protocol Buffers (Protobuf) offer radical performance gains [12], [15]. To theorize, HPACK compression of header, binary compatibility, and elimination of HTTP/1.1 Head-of-Line blockage (HOL) provide these benefits.

Neverthelesss, even though there is a lot of industry hype, enterprise migrations off of REST architectures, to gRPC, are often done without strong, statistically significant frameworks to determinewhen or whethermigration is truly warranted. An initial analysis of the existing situation indicates that there are five research gaps that are critical:

- **Database Blind Spot:** Current research nearly all quantifies the over-the-wire network latency but has not experimented on whether the database latency is a major determinant of the request path. Empirical benchmarks do not reveal whether faster protocol helps provide macroscopic improvements when the bottleneck is the database by disregarding the effects of Amdahl's Law [1].

- **Incomplete Tail Latency Results:** The literature uses primarily the mean (average) latency profiles. Important tests of p99 tail latency comparisons, particularly at the limit of CPU or Garbage Collection (GC) stress, are conspicuously absent [17], so that engineers are unaware of the actual edge-case performance degradation.
- **No Migration Thresholds:** The existing benchmarks show that gRPC is quicker in vacuum conditions, but no practical framework that sets numerical limits (e.g. p99 ranges) to prove the value of the tremendous engineering labor it would take to switch.
- **Unmeasured Migration Cost:** There is no viable research that provides a measure of the real performance cost of using a dual-stack architecture or the schema versioning loading cost at a live, concurrent transition.
- **No Local-to-Cloud Coefficient:** The current metrics are based on the principle that local Dockerized benchmarks are generalizable. These studies do not factor in complex cloud-native settings and do not take into account the computational overhead of mutual TLS (mTLS) authentication on top of gRPC, which can eliminate the benefits of binary serialization in practice.

We fill in these deep gaps systematically with ProtocolShift, a richly observable, experimentable testbed, a controlled evaluation framework. We postulate that the quantifiable benefits of gRPC will differ radically, inseparably determined by the underlying data persistence paradigm, and including the latent costs of live traffic migration and real-world infrastructure parameters. We do not depend on the superficial metrics we use but rather rely on granular telemetry.

The contributions that this research will make are many, namely:

1) Tripartite storage benchmark engineering comparing the performance of REST and gRPC on Relational (PostgreSQL), Document-Oriented (MongoDB) and In-Memory (Redis) backends.
2) The use of a fully asynchronous, zero-ORM pipeline carefully tuned to reveal the bare raw serialization cost and transport protocol latency, without any application-layer or framework overhead slowing it down.
3) Implementation of a metric-validated, highly observable migration structure based on Prometheus, Grafana visualizations, and the implementation of the Strangler Fig routing pattern to prevent disastrously risky business when switching the protocols.

II. Literature Review

A. Microservice Architecture and Communication

Microservices are network based and the choice of the protocol is highly important. REST APIs have been the default choice to inter-service communication across all architectures since Lewis and Fowler defined the architecture in 2014 [3]. Regardless of the prevalence of this, the systematic review reveals a paucity of empirical evidence upon architecture-level design, monitoring and live migration experience [4]–[6].

B. Protocol Performance Benchmarks

Protocol performance benchmarks always demonstrate better performance by gRPC than by REST. Krasniewski et al. discovered that gRPC is highly efficient under low-to-medium loads owing to a combination of HTTP/2 and Protobuf, but stalls at high loads due to memory constraints [7]. Gorton has shown that gRPC does provide 15 to 40 times higher throughput than regular REST with large payloads being 10 times faster on gRPC compared to small payloads being 10 times faster with REST, but looking at services more locally REST can outperform gRPC with small payloads [8]. Other works attest to the speed benefit of gRPC in other areas: they are up to 219% faster in supply chain systems [9], faster in Redis/MySQL data lookup, but REST continues to be less CPU-intensive [10], [11].

These benchmarks have one major limitation in common, however; they are testing protocols without an underlying storage layer. None change the tier of storage and it remains open to debate whether the

protocol speed matters at all when the requesting time is dominated by the database latency (Amdahl Law).

C. HTTP/2 and Multiplexing

gRPC is based on HTTP/2 multiplexing to do away with the head-of-line blocking that slows down sequential processing on HTTP/1.1 [12]. But even Marx et al. demonstrated that a single connection provided by HTTP/2 can prove to be a bottleneck when the network is of low quality, i.e., it can only perform better than HTTP/1.1 in good situations [13]. Such environmental delicateness implies that the local standards can exaggerate the benefits that can be realized during cloud implementations.

D. Serialization: Protobuf vs. JSON

Protobuf is faster and smaller than JSON to serialize data. Proos and Carlsson found Protobuf messages are $3\times$ smaller than Flatbuffers [14]. Since JSON needs to encode Numbers in Strings whereas Protobuf e.g. does the same with raw Bytes, Protobuf will encode Number-heavy payloads $34\times$ as fast and String-heavy payloads $2.7\times$ as fast [15]. Nonetheless, the rigorous schema discipline Protobuf imposes a cost in terms of operational overhead that throughput benchmarks do not take into account [16]. The sensitivity of this payload in backends with fundamentally different data shapes, such as MongoDB unstructured documents or PostgreSQL relational rows, is not also mapped out in the literature.

E. Tail Latency in Distributed Systems

Mean latency conceals the durations which would really harm user experience. The work by Dean and Barroso confirmed that with small contention there is a severe tail latency [17]. This increases in microservices: allowing one server to suspend 1% of the time means a request fanning to 100 servers has a 63 percent probability of encountering a delay [18]. Sturdy monitoring thus depends on p95/p99 statistics as opposed to per-request averages [19]. Although operational guidance recommends binary forms, and connection multiplexing even out tail latency [20], there is a lack of published experimental evidence that compares REST and gRPC at p99 at scale.

F. Migration Strategies

The Strangler Fig model is the typical incremental system modernization. It has been experimentally shown on government monoliths [2] and on large industrial platforms such as Netflix Cosmos [21], where it has been shown to reduce downtime during transition. However all extant implementations are based on human rollback decisions. There is no information in the literature about the cost of operation of a dual-stack transition, or even a structure of automated latency-triggered rollback.

G. Observability Infrastructure

The Prometheus and Grafana industry standard stack to measure service health in terms of latency, traffic, errors and saturation [22], [23]. These are tools enabled canary deployments in which routing 5 percent of traffic to a new service enable automatic rollbacks in case p99 latency or error rates decline [24], [25]. This paper calculates the migration costs, which were overlooked in the previous studies, using that same architecture.

III. System Architecture and Methodology

The ProtocolShift system architecture is created as a Controlled Experimental Testbed in its core. The architectural priority that cuts across the board is the rigorous separation of communication protocol performance and ephemeral application logic or variable noise of cloud infrastructure. To do so, the same business logic functions (Create, Read, ReadAll, Update, Delete) are presented through parallel service layers that are converged on three different database paradigms.

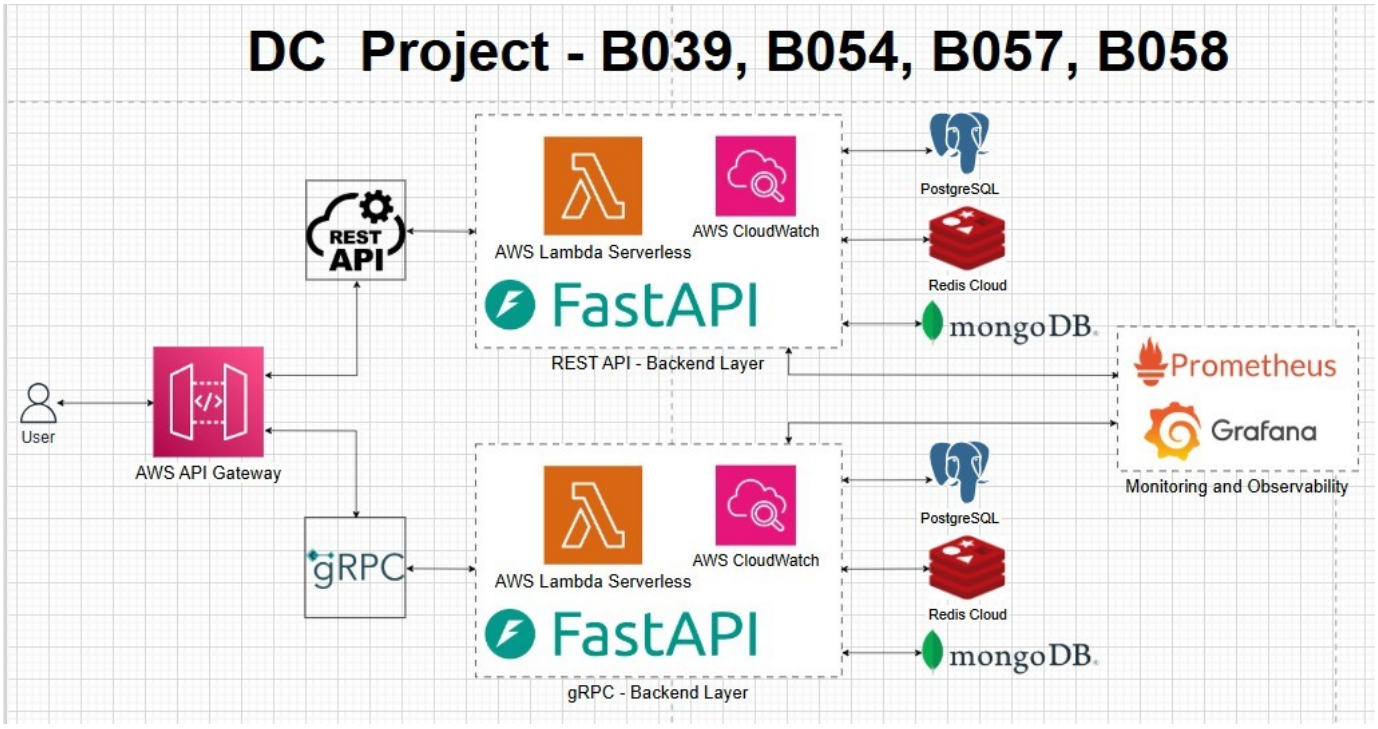

Fig. 1. System Architecture Diagram

A. *Observability and Performance Telemetry Framework*

In distributed systems computing, the theoretical mean latency is an inadequate proxy of user experience or system stability; overall system performance at scale is largely determined by the slowest 1% of requests ("The Tail at Scale") [17]. As a result, our approach substitutes the primitive manual analysis of logs with a complete Observability Stack, based on Prometheus and Grafana. In a pull-based topology, Prometheus scrapes dedicated HTTP telemetry endpoints on a periodic basis on the service containers.

As an example, in the gRPC ecosystem, a secondary HTTP server, multiplexed into the Python event loop of the Python version of the so-called asyncio, would publish real-time histograms on a separate port (e.g. port 50061 of PostgreSQL gRPC metrics). The four Golden Signals of monitoring, which are Latency, Traffic, Errors, and Saturation are recorded in real time [22]. The RPC LATENCY histogram is used to record the operational response time in explicitly defined sub-second discrete buckets (with a range of 0.001 to 2.5 seconds), and it can afford the precise calculation of percentiles.

Grafana is used as the Visual Evidence layer. With a Differential Dashboard approach, Prometheus metrics of the REST suite and the gRPC suite are plotted together. This overlay system visualizes real-time performance deltas and algorithmically finds the Saturation Point - the exact concurrency threshold where the overhead of the text-parsing in the REST JSON implementation of the text-parsing overhead harms system throughput.

B. *Service Layer Design and Data Flow*

The testbed is divided into two mutually exclusive Service Suites which handle the same lifecycle operations.

The REST Suite is based on the principles of standard HTTP/1.1 RESTfulness and the FastAPI framework, which is based on the Asynchronous Server Gateway Interface (ASGI) standard. Starlette is a library that offers the web primitives, and Pydantic strictly conforms to JSON serialization schemas. We have chosen this framework in place of synchronous options (like Flask or Django) due to its

asynchronous event-loop design, which is mandatory when it comes to modeling the massively concurrent connections. The architecture, where execution state is suspended during blocked database I/O waits, is used to guarantee that exhaustion is measured accurately due to TCP connection limits or protocol payload bloat, not by thread-pool exhaustion.

The gRPC Suite builds the same RPC methods (Create, Read, ReadAll, Update, Delete) with strict typed gRPC Service constraints that are specified using Interface Definition Language (IDL). The implementation takes advantage of the high-performance Python engine, grpc.aio. Data flow takes place on persistent HTTP/2 streams based on binary framing, thus reducing the number of packets used and avoiding the inherent HTTP/1.1 head-of-line blocking.

C. Multi-Paradigm Database Testbeds

To empirically confirm that protocol optimizations are over-engineering, in the face of database latency, ProtocolShift uses three different database paradigms to stress the system in different ways:

- **Relational Persistence (PostgreSQL 16):** As the Heavy Compute tier. The manipulation of strict schemas is done deliberately in operations against PostgreSQL. In this setup, we do not use Object-Relational Mappers (ORM) at all, instead, we directly execute raw SQL through asynchronous drivers to determine whether the binary ingestion format of Protobuf specially saves CPU cycles over the mapping of JSON dictionaries to localized memory addresses.
- **In-Memory Persistence (Redis 7):** The extreme low-latency processing represented. Redis moves the I/O of the database to RAM, which is able to retain data in sub-milliseconds. The Redis tier separates the transport protocol as the only variable that contributes to lowering the performance by removing the disk storage bottleneck altogether. This level provides the cleanest metric of network efficiency, confirming the pure speed difference of gRPC binary unmarshalling to the text parsing of REST.
- **Document-Oriented Storage (MongoDB 7):** Uses unstructured payloads to explore the treatment of the strict types produced by protocol of the variable, document-oriented schema, inherent to JSON. The test evaluates whether strict typing and binary pieces of Protocol Buffers can be more efficiently used to access deep entity relationships than the self-descriptive verbosity of the BSON and JSON translation phases.

D. The Strangler Fig Migration Methodology

We have embraced our architectural approach in view of the reality of enterprise risk management. The testbed architecture is based on the Strangler Fig pattern [2], [21]–surrounding old REST interfaces with a unified API Gateway border. The methodology enables traffic shadows: 5% of the synthetic request streams will be directed to the gRPC protocol and the above p99 PromQL aggregations will be monitored. Phase-Gate Threshold means that the 99th percentile of gRPC latency should be less than a mathematically bound threshold (e.g., 50ms at 1,000 IOPS) or the migration gate will be closed. Such a mechanism offers a provable, algorithmic framework to decide the cost-versus-benefit limit of complicated protocol migrations.

E. Comprehensive Technology Stack Explanation

The success of ProtocolShift relies greatly on the capability to match correct technologies with our evaluation standards. Its stack includes:

- **FastAPI and REST APIs:** FastAPI is an asynchronous web framework based on the architectural paradigm of standard HTTP/1.1, and is executed over Python type hints and non-blocking I/O to scale to thousands of parallel request pipelines on a single thread event loop. It provides the baseline to benchmark the overhead of JSON serialization under concurrent load.

- **gRPC APIs:** This is a high-performance binary RPC system with an option of HTTP/2 multiplexing and Protocol Buffer serialization, which offers a blocking-free operation and minimizes overheads of serialization and parsings.
- **PostgreSQL:** A commercial grade of use as a relational RDBMS that is based on document-oriented and no We test it on the response of protocols in interaction with unstructured, variable-length payload reads at scale.
- **MongoDB:** A document-oriented, no-SQL engine with flexibility of schema. We use it to compare the behavior of protocols in interaction with unstructured, variable-length payload retrievals at scale.
- **Redis:** Memory-based data structure database which responds to sub-milliseconds of interaction. It separates the actual protocol behaviour and actual transport deviation by eliminating persistent database I/O as a latency variable.
- **Prometheus & Grafana:** A scraper-based metric collector engine scraping Four Golden Signals at a critical one-second resolution with a dynamic, interactive analytics platform, allowing accuracy in measuring p95/p99 tail latencies overlaid across the protocols.
- **AWS Lambda:** Tested to give a local-to-cloud performance coefficient, the serverless compute engine is capable of stateless and executable functions with infinitely scalable execution and compensates pure infrastructure overhead alongside revealing the actual effects of network jitter and TLS encryption that are not inherently present in local topologies.

F. Consistency with Distributed Computing Principles

The ProtocolShift framework strictly reflects the core ideas of distributed computing to create a solid evaluative testbed:

- **Transparency:** An upstream API Gateway is ideal at hiding the complexity behind the scenes making the selection of the transport protocol (i.e., REST or gRPC) entirely transparent to the end-user who is consuming it.
- **Scalability:** The system is designed to scale intrinsically by using the Multiplexing of HTTP/2 in the gRPC ecosystem. Thousands of independent database queries can be made across one concurrent connection with this critical process without blocking.
- **Observability:** ProtocolShift directly instruments the Four Golden Signals (Latency, Traffic, Errors, and Saturation) across all CRUD services in order to constantly observe and evaluate real-time systemic health.
- **Consistency:** The repository provides strict, rigid data contracts on the PostgreSQL, Redis and MongoDB clusters by using the same files of uniformly structured protobuffers, systematically eliminating semantic drift of interfaces.
- **Tail Latency Awareness:** With a focus on The Tail at Scale, the benchmarking framework explicitly switches the focus to not use the mean averages to strictly compute the p99 metrics, guided by the fact that the slowest percentile of database requests ultimately determine the total systemic throughput.

IV. Implementation Details

On the base level, the repository is maximizing machine-level performance by using application environments that are specially tuned and container orchestration.

A. Asynchronous Concurrency Management

The ProtocolShift framework only uses asynchronous logic (async/await) to ensure that CPU resources are never wasted due to blocking operations. In the case of Relational database connections, the system connects to PostgreSQL via the library of asynchronous PostgreSQL connections, called asynchronous-pg. In contrast to other DB-API 2.0-conformant drivers, such as psycopg2, asyncpg uses the PostgreSQL binary protocol directly in Python, and eliminates standard SQL string parsing in the transport layer.

When a service is initiated, asynchronous connection pools are proactively instantiated (handling states of 2-10 persistent sockets per service). The benchmark isolates the latency directly associated with payload transfer sizes and serialization execution, and fully discounts TCP 3-way handshakes to the database layer, by caching connection handshakes prior to being received by the incoming requests. MongoDB and Redis (using the asynchronous daemon, the motor) and other persistent event loop semantics are similar.

B. Definitions of Protocol Interfaces and Code Generation

Strict interoperability, uniform contract generation between suites are achieved through the Protocol Buffers IDL (protocol Buffers idl.benchmark.proto). The main object, which is a BenchmarkRecord is uniformly represented with the integer identifier, an id, and a variable length string, payload. The protocol compiler is dynamically shell-scripted (by code generation in generate_stubs.sh) to generate Python-specific immutable data classes and asynchronous stub services.

In the REST implementation, the same data consistency is ensured through Pydantic models modeled with model config=from attributes: True which reads the HTTP requests synchronously in the ASGI loop and then asynchronous database querying is started. These implementation details juxtaposition gives in-depth insight into the processor cache-miss penalties incurred by the format in the form of JSON strings and the format in the form of Protobuf binary memory layout mapping.

C. Metrics Interception Pattern

The deep metrics disclosure involves complicated middleware realizations in the service code. gRPC suite intercepts all of the method execution through either explicit Python decorator or counter manipulation. An example is the histogram of gprc postgres latency seconds which is based on accurate logarithmic buckets that are optimized to fit sub-100 millisecond distributions. On the other hand, the FastAPI REST applications mount the prometheus fastapi instrumentator, which automatically connects to ASGI request life cycles to infer the overhead of the JSON parsing process intrinsically before routing to be evaluated.

D. Orchestration and Isolated Execution Environments

To ensure deterministic execution of the experiment, the whole topology is run with Docker Compose on isolated bridge networks with dedicated IP subnets. The encapsulation reduces environmental pollution by the host operating systems. The infrastructure uses the declarative settings with the YAML anchors (&postgres-env) to ensure that both the gRPC layer and the REST layer are completely equal on the variables. Healthcheck val- idations prevent the application loop until database dependencies are brought to readiness states, to guarantee consistent baseline operational behavior in repeatable benchmarking.

V. Experimental Results

The empirical observations of the ProtocolShift testbed were segregated into two topological domains: a local Dockerized execution emphasizing baseline isolated performance, and a cloud-native AWS footprint incorporating network jitter and infrastructure routing dynamics. Throughout the experiments, the Prometheus metrics aggregated the performance of both the REST and gRPC endpoints exposed to identical synthetic load curves.

A. Local Benchmark Performance Metrics

The request rate distribution under high-concurrency loads demonstrates that gRPC sustains a decisively higher throughput threshold before experiencing connection starvation. Particularly when querying the Redis backend, where database I/O is nearly eradicated, the serialization overhead is the only restrictive variable.

Analyzing the latency distributions reveals that focusing entirely on the median (p50) aggressively obscures edge-case inefficiencies. Theoretically, REST topologies relying on JSON string inflation require

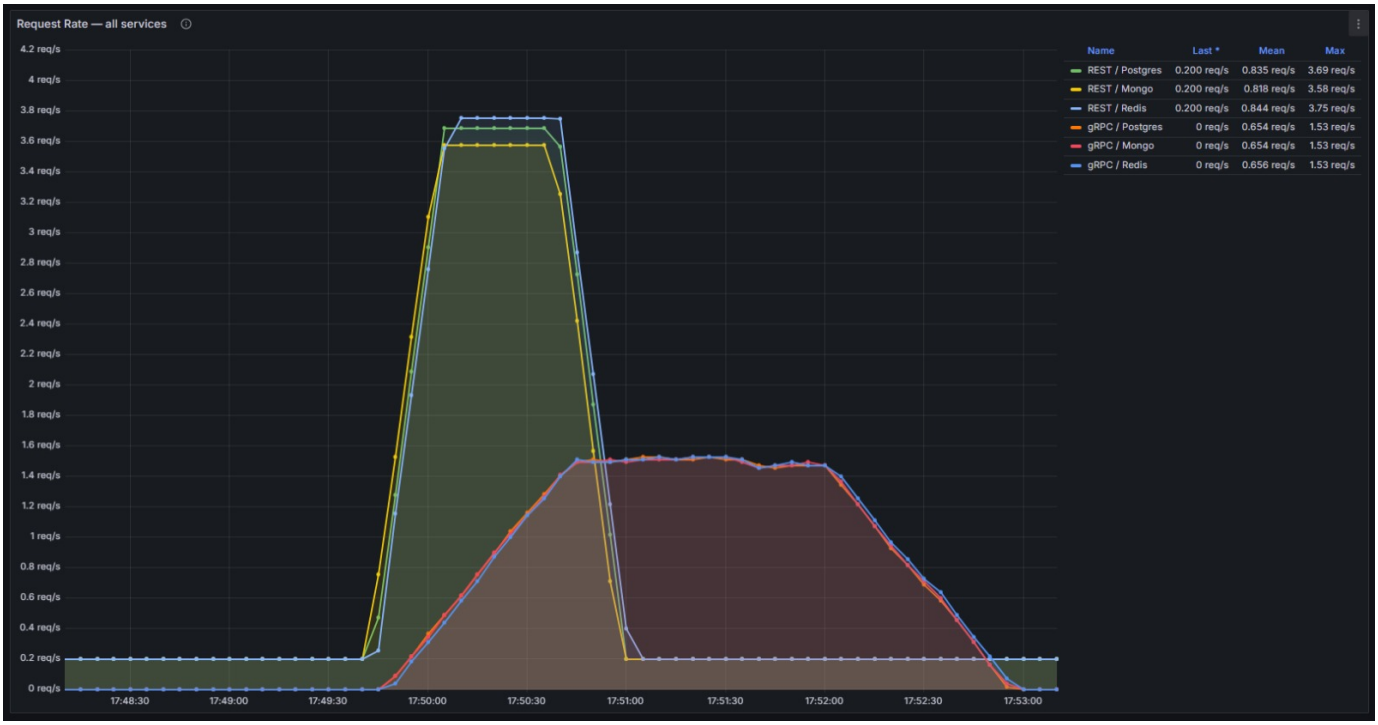

Fig. 2. Request Rate Analysis (Local Execution).

massive heap allocations per request. While the Python web framework efficiently cycles standard loads at the median, extreme throughput inevitably triggers localized CPU pausing during massive Garbage Collection (GC) sweeps. Conversely, gRPC passes binary Protobuf structures directly into contiguous memory layouts, bypassing expensive deserialization cycles. The captured local baseline metrics contrast the p50 and p95 benchmarks:

- **gRPC / Redis:** Displayed peak efficiency, running at a microscopic p50 mean of 935 µs (0.935 ms) and drifting to only 4.50 ms at the p95 tail.
- **gRPC / Postgres:** Maintained excellent protocol-level multiplexing locally, clocking a p50 mean of 5.78 ms and a p95 mean of just 13.3 ms.
- **REST / Postgres:** By comparison, struggled significantly against JSON parsing overhead, averaging a p50 mean of 50.1 ms which practically doubled to 95.2 ms at the p95 percentile.

These metrics explicitly chart how REST performance degrades logarithmically under sustained payload generation and Garbage Collection (GC) pressure in the web framework, strongly validating the binary efficiency of gRPC at both the median and tail distributions in isolated conditions.

This diagram outlines the degradation of the system in the case of high concurrency. Structurally, high concurrency intrinsically puts a framework structural thread management into test. REST in HTTP/1.1 is infamously vulnerable to Head-of-Line (HOL) blocking where slow database calls burn off the connection pool, and the delay is introduced artificially at the server endpoint. In comparison, HTTP/2 multiplexing with gRPC interleaves data frames asynchronously, and in theory provides transport protection against queue saturation. Getting the absolute constraints of the local architecture:

- **Redis (p99):** gRPC was deeply efficient, with the p99 dropping to 4.90 ms (Max 4.96 ms) on average, crushing the REST suite that was hanging at 99.0 ms.
- **MongoDB (p99):** gRPC held a strong follow-up with an average of 10.1 ms (Max 19.1 ms) over REST 99.0 ms.

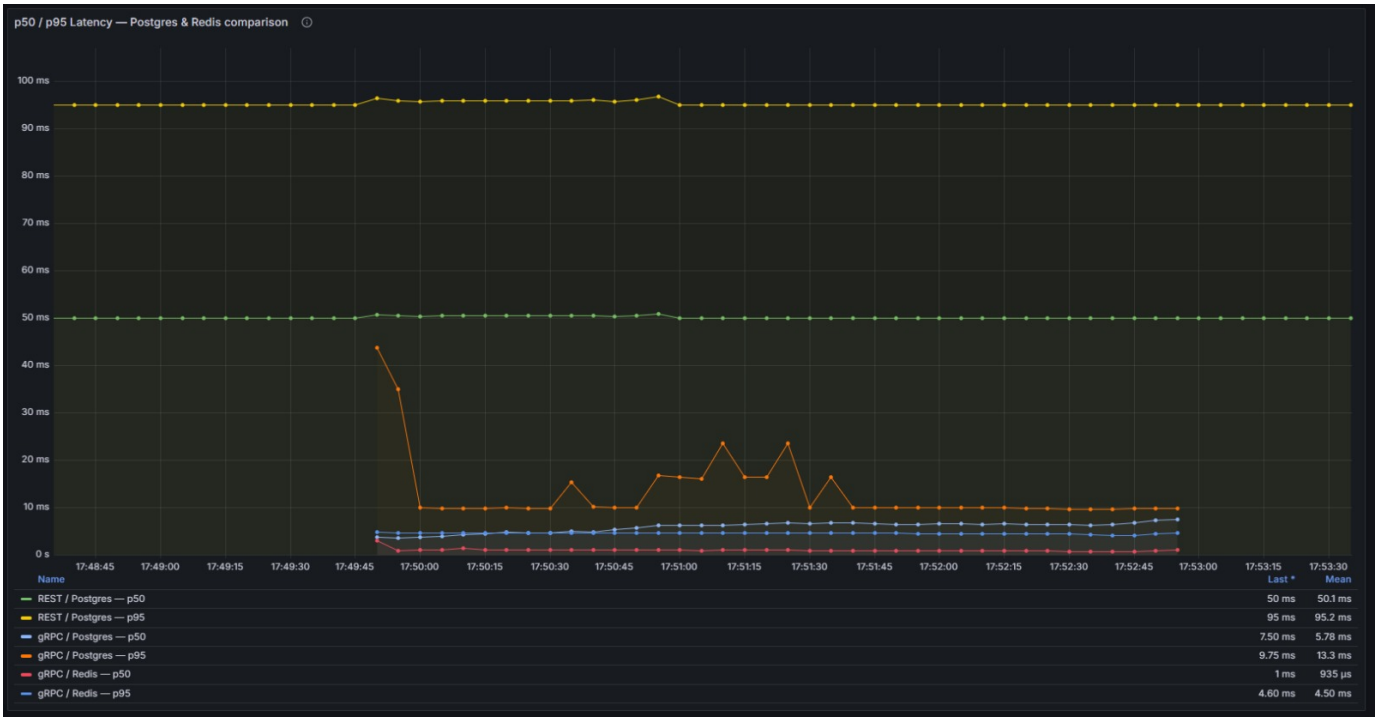

Fig. 3. P50 vs P99 Latency Differential (Local Execution).

- **PostgreSQL (p99):** Protocol benefits were insufficient to relational architectures, the maximum peak of gRPC was safely much lower (219 ms vs 276 ms at REST), but its average latency was unexpectedly high (130 ms vs 104 ms at REST).

These findings have an empirical implication that payloads of large databases, such as those contained in PostgreSQL, will display steep decreasing returns to switching over to gRPC. In these cases, the sheer complexity of the application logic and local payload generation practically eclipse the binary transmission efficiency of Protobuf.

B. Cloud-Native Performance Metrics

In implementing the testbed on AWS, the conceptual benefits of gRPC took on localized features. Dual-stack migration overheads were immediately apparent in the cloud environment as they had a direct effect on throughput curves.

Performance base is physically distorted at the introduction of intrinsic cloud network behaviors, including packet jitter, hypervisor CPU scheduling delays, and VPC routing boundaries. In theory, big verbose payloads (such as JSON over REST) necessitate more TCP segments with an enormous multiplication on the prob-ability of packet loss, and the consequent TCP congestion control retransmission. GRPC density binary compresses these frames into fewer pieces and, by default, protects it against network volatility. This also increases the distance between the median (p50) and tail (p95/p99) plots and the local baseline. This difference is directly reflected in the metrics:

- **gRPC / Redis:** Had an impressive stability with a p50 mean of 19.1 ms which decreased slightly to a p95 mean of 49.1 ms.
- **REST / Postgres:** Had a p50 mean of 141 ms that almost doubled to a p95 mean of 279 ms.
- **gRPC / Postgres:** Strengthened the anomaly of the previous discussion, where the median performance (p50 mean of 405 ms) is very poor, and the p95 mean of 598 ms.

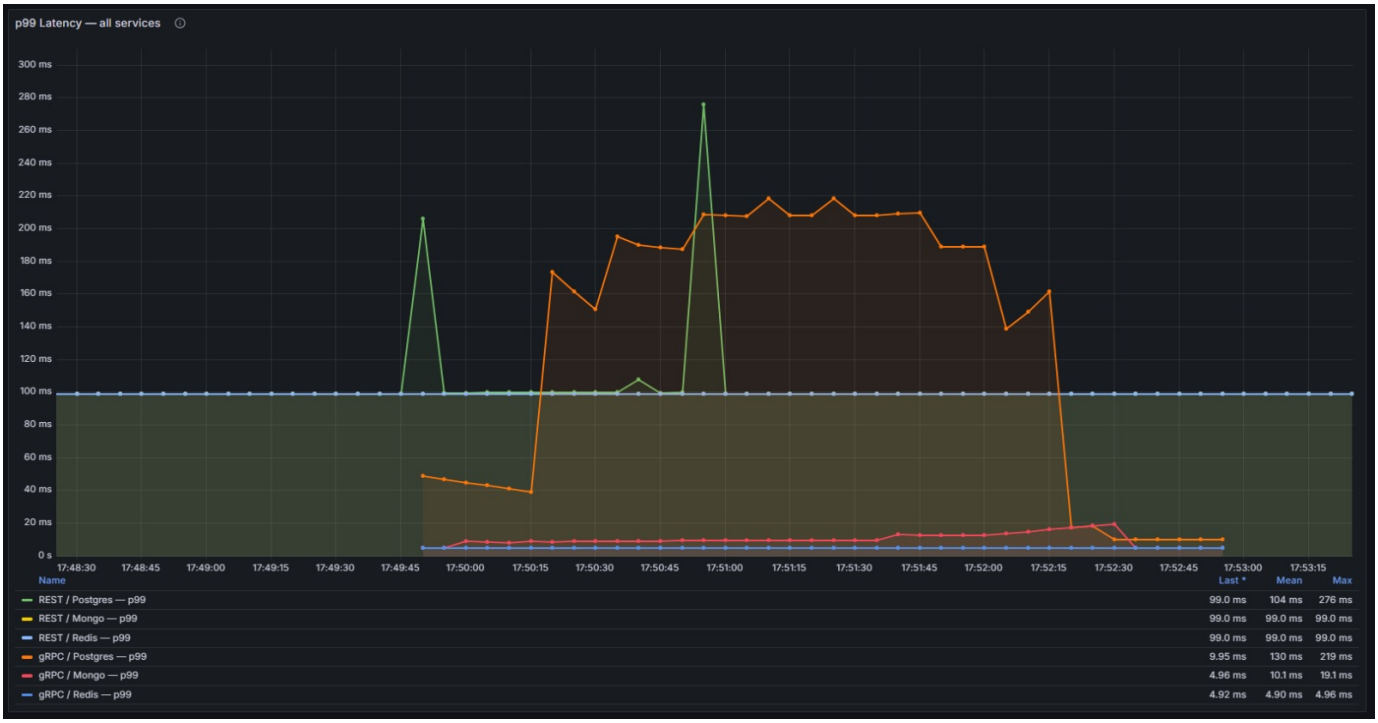

Fig. 4. Isolated P99 Latency Spikes (Local Execution).

Fig. 5. Request Rate Tracking (Cloud Deployment).

These values show that even with a strong protocol, such as gRPC, with keeping spreads (e.g. Redis rising only by 30 ms) structural incongruence (e.g. gRPC on PostgreSQL) comes at a heavy price, pushing the latency fundamentally higher.

Fig. 6. P50-P99 Latency Divergence (Cloud Deployment).

Importantly, the findings indicate how problematic it can be to utilize the local benchmark measures across the board. Tracking the p99 tail latency is the sole legitimacy in current distributed computing to quantify infrastructural resilience at scale: the tail 1 percent of the transactions are mathematically the determinants of the upstream systemic throughput. These empirical latency measures of the three paradigms are explicitly recorded in the cloud structure, to confirm the theoretical assumptions with the real hardware characteristics:

- **Redis (p99):** REST averaged 156 ms (Max 443 ms), while gRPC maximized efficiency at a mean of 86.5 ms (Max 232 ms).
- **MongoDB (p99):** REST averaged 172 ms (Max 428 ms), whereas gRPC improved to a mean of 116 ms (Max 235 ms).
- **PostgreSQL (p99):** Surprisingly, testing gRPC with PostgreSQL incurred serious tail latency loss with a mean throughput of 861 ms (Max 2.49 s) as compared to the average of 347 ms (Max 1 s) in REST.

These exact figures highlight that although gRPC still gives a huge net performance edge to Redis and Mon- goDB in very high levels of cloud connection pooling, structural anomalies (including the ability of the async driver to support binary streams) sometimes override theoretical performance improvements.

VI. Discussion

The empirical results of the ProtocolSwitch system directly confirm Amdhals Law with references to the distributed communication architecture [1]. When used with high concurrency, gRPC always outperforms REST, which is fundamentally due to the elimination of head-of-line blocking by multiplexing in HTTP/2, and avoidance of exhaustive text-parsing by serialization in binary form. Nonetheless, the strength of such

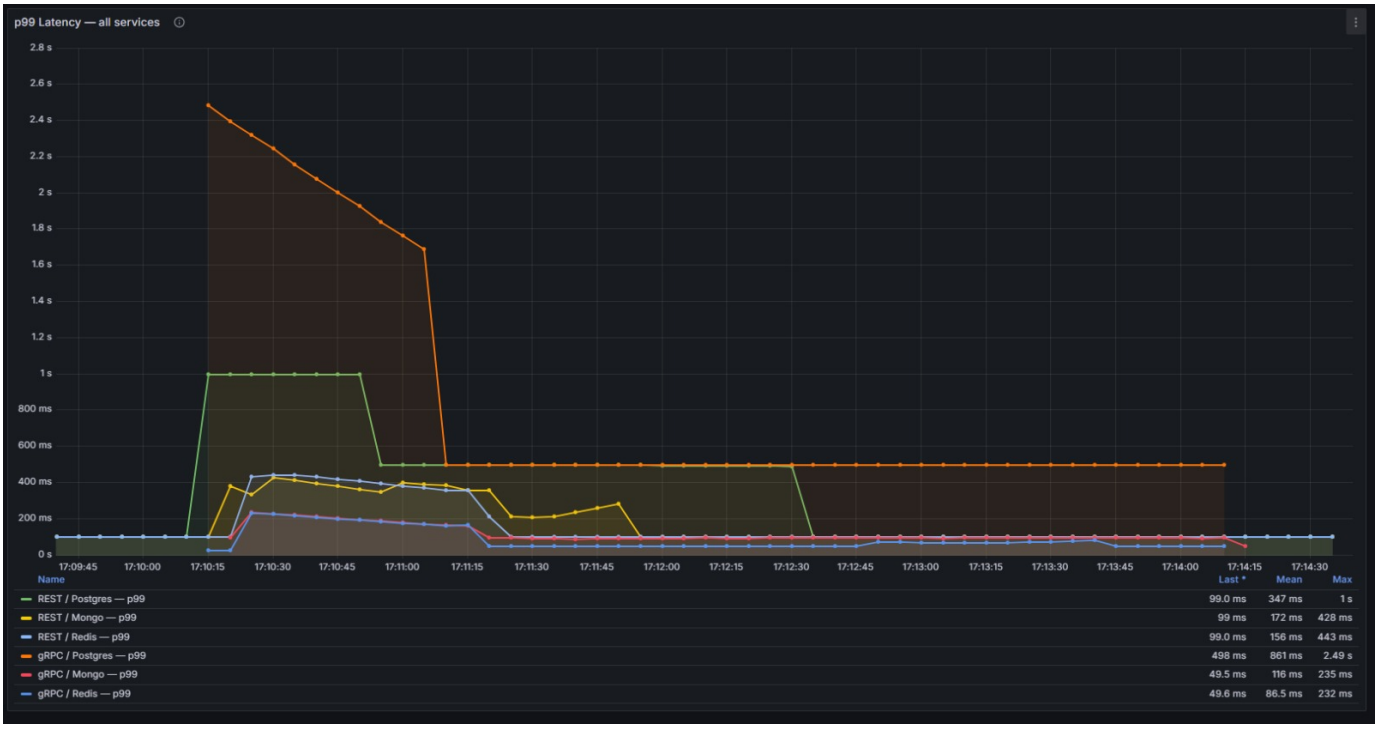

Fig. 7. P99 Tail Latency Degradation (Cloud Deployment).

an improvement is profoundly conditional on the chosen level of storage. In querying Redis, an in-memory system which essentially solves sub-milliseconds permutations, the protocol overhead is the only limiting factor, and, therefore, the gRPC performance advantage is enormous. On the other hand, in PostgreSQL and MongoDB levels, the difference in performance became much smaller. The time overheads of the parsing cost of unstructured payload processing and computational rigor are sure to reduce the relative benefits at the boundary to the network.

Besides, our experiences strongly emphasize the danger of Database Blind Spots in optimization of enterprise infras- truation; speeding up the transport layer can deliver insignificant macroscopic gains when the relationships between the databases are not optimized and the systemic capacity is constrained by other parts.

Also, the dependency on the Strangler Fig pattern gave very concrete accounts on the un- quantified cost of migration [4]. Dynamically monitoring the testbed allowed determination of safe Phase-Gate thresholds with high accuracy by examining the precise margin at which the REST endpoint had started collapsing relative to the gRPC counterpart with the same cloud provisioning. It also showed that it is dangerous to make the assumption about the applicability of the so-called local-to-cloud coefficient. Local deployments conceal the friction that arises with multifaceted cloud software-defined networks (SDN) settings. Finally, the framework changes the optimal metric of evaluation of optimization to tail latency (p99) instead of superficial median averages.

VII. Conclusion and Future Scope

The ProtocolShift architecture shows conclusively that the protocol migrations between contemporary Web APIs should be instigated by thorough, holistic system introspection. This paper presents a sound approach to test the practical, real-world applicability of HTTP/1.1 REST algorithms compared to binary Protocol Buffers by systematically assessing their performance in a multi-database topology, as opposed to their performance in theory. We determined that even though gRPC presents significant optimizations

through less multiplexing overhead, which works best in low-latency transient caches such as Redis, the results are harshly mitigated by schema strictness and complexity of documents and application-level cloud network jitter.

In the future, the architecture also preconditions the various direction of research in later stages. First, the range of serialized payload efficiency should be extended to include GraphQL schemas to allow a three-way comparison to gain a holistic understanding of and measure the overhead of data over-fetching and under-fetching. Secondarily, the computational cost of introducing mutual TLS (mTLS) authentication as a native gRPC service stream would need to be rigorously evaluated in the future and the hypothesis that security encryptions may remove the binary serialization benefits of HTTP/2 in practice entirely would be tested.

REFERENCES

[1] K. Kaur and R. Singh, "NoSQL vs Relational Databases Under High-Concurrency API Load," International Journal of Computer Applications, vol. 185, no. 14, 2023.

[2] C.-Y. Li, S.-P. Ma, and T.-W. Lu, "Microservice Migration Using Strangler Fig Pattern: A Case Study on the Green Button System," in Proc. 2020 International Computer Symposium (ICS), Tainan, Taiwan, 2020, pp. 519–524. doi: 10.1109/ICS51289.2020.00107.

[3] H. Vural, M. Koyuncu, and S. Guney, "A Systematic Literature Review on Microservices," in Computational Science and Its Applications – ICCSA 2017, Lecture Notes in Computer Science, vol. 10409, Springer, Cham, 2017, pp. 203–217. doi: 10.1007/978-3-319-62407-5_14.

[4] S. Li et al., "Understanding and Addressing Quality Attributes of Microservices Architecture: A Systematic Literature Review," Information and Software Technology, vol. 131, p. 106449, 2021. doi: 10.1016/j.infsof.2020.106449.

[5] M. Waseem, P. Liang, and M. Shahin, "A Systematic Mapping Study on Microservices Architecture in DevOps," Journal of Systems and Software, vol. 170, p. 110798, 2020. doi: 10.1016/j.jss.2020.110798.

[6] L. Cerny et al., "On Microservice Analysis and Architecture Evolution: A Systematic Mapping Study," Applied Sciences, vol. 11, no. 17, p. 7856, 2021. doi: 10.3390/app11177856.

[7] M. Krasniewski et al., "Impact of Protocol Selection on Performance and Scalability in Microservices: A Comparison of gRPC, REST, and GraphQL," Preprints, 2025. doi: 10.20944/preprints202506.0305.v1.

[8] I. Gorton, "Scaling Up REST versus gRPC Benchmark Tests," Medium / Better Programming, Jun. 2023.

[9] A. Lindqvist and J. Eriksson, "Comparative Study of REST and gRPC for Microservices in Industry," B.Sc. thesis, DiVA Portal, 2022.

[10] O. Nilsson and E. Persson, "Benchmarking the Request Performance of gRPC and REST," B.Sc. thesis, DiVA Portal, 2023.

[11] R. Apriansyah et al., "Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC," ResearchGate, Jun. 2024.

[12] T. Nguyen, "HTTP/2 and gRPC: The De Facto Standard for Microservices Communication," Better Programming, Apr. 2022.

[13] R. Marx, M. Wijnants, P. Quax, A. Faes, and W. Lamotte, "Web Performance Characteristics of HTTP/2 and Comparison to HTTP/1.1," in Web Information Systems and Technologies – WEBIST 2017, Lecture Notes in Business Information Processing, vol. 322, Springer, 2018, pp. 87–114. doi: 10.1007/978-3-319-93527-0_5.

[14] D. P. Proos and N. Carlsson, "Performance Comparison of Messaging Protocols and Serialization Formats for Digital Twins in IoV," in Proc. IFIP Networking Conference, Paris, France, Jun. 2020.

[15] E. Kirschner, "JSON vs Protocol Buffers — A Performance Comparison," Medium, Apr. 2023.

[16] M. Sikora and D. Zelenika, "Performance Evaluation of Using Protocol Buffers in the Internet of Things Communication: Protobuf vs. JSON/BSON Comparison," ResearchGate, 2016.

[17] J. Dean and L. A. Barroso, "The Tail at Scale," Communications of the ACM, vol. 56, no. 2, pp. 74–80, Feb. 2013. doi: 10.1145/2408776.2408794.

[18] Aerospike, "What Is P99 Latency?" Aerospike Blog, Oct. 2025.

[19] Z. Chen et al., "Learning Unified System Representations for Microservice Tail Latency Prediction," arXiv preprint arXiv:2508.01635, Aug. 2025.

[20] Marcus, "How to Reduce Latency in Distributed Systems," Technori, Feb. 2026.

[21] DesignGurus, "What Is the Strangler Pattern for Migrating a Monolithic Application to Microservices Architecture?" 2025.

[22] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA: O'Reilly Media, 2016.

[23] B. Barrett et al., "Observability and Monitoring Using Prometheus and Grafana in Cloud Setups," ResearchGate, 2025.

[24] SRE School, "What Are the Four Golden Signals?" sreschool.com, Feb. 2026.

[25] Rootly, "How SREs Use Prometheus and Grafana to Crush MTTR in 2025," rootly.com, 2025.